La capa de Transporte

Principios de transferencia de datos fiable

Tema 3: La capa de Transporte.

3.1 La capa de Transporte y sus servicios.

3.2 Multiplexacción y demultiplexación.

3.3 Transporte sin conexión: UDP.

3.4 Principios de un Servicio de transferencia de datos fiable.

3.4.1 Construcción de un protocolo de transferencia de datos fiable (resumen).

3.4.2 Protocolo de transferencia de datos fiable con procesamiento en cadena.

3.4.3 Retroceder N (GBN).

3.4.4 Repetición Selectiva (SR).

3.5 Transporte orientado a conexión: TCP.

3.6 Principios de control de congestión.

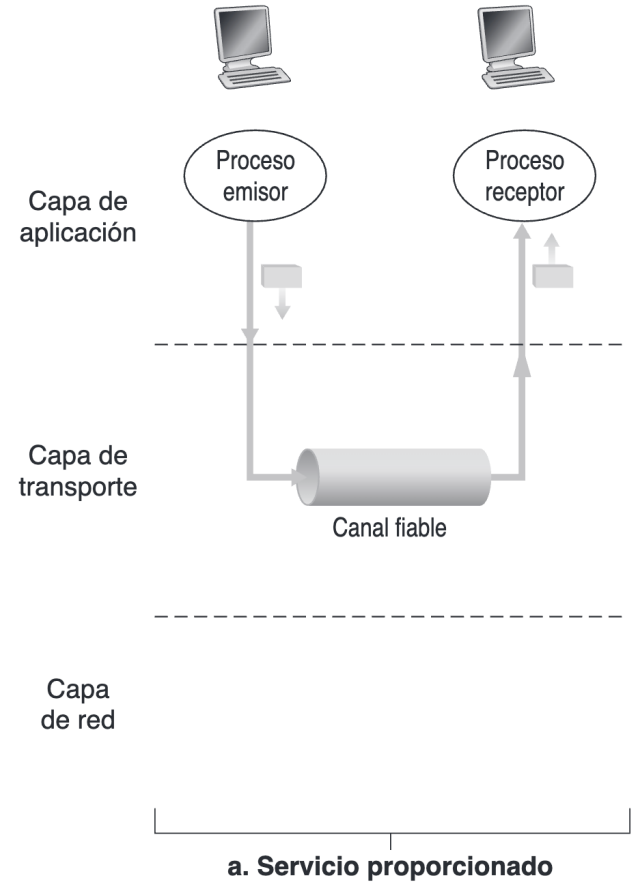
3.7 Mecanismo de control de congestión de TCP.

Transferencia de datos fiable: principios generales.

- Problema muy relevante en Redes de Computadores.
- En principio aplicable a cualquier capa.
  - En la arquitectura de protocolos TCP/IP se puede dar en:
    - Capas de Aplicación, de Transporte y de Enlace.
- Modelo de servicio
  - No bits corruptos.
  - No pérdida de bits.

  } Abstracción de un canal fiable
- Por simplificación:
  - Análisis de transferencia de datos UNIDIRECCIONAL.

Transferencia de datos fiable: abstracción.



- Protocolo para transferencia fiable de datos en capa "N".
  - Complicado si capa "N–1" brinda servicio NO fiable.
  - Sencillo si capa "N–1" brinda servicio fiable.
- En la figura: abstracción para provisión de un servicio "rdt" (rdt, **r**eliable **d**ata **t**ransfer).
  - P.e: rdt en capa 4 (capa de Transporte).

Transmisión:
• De paquetes de datos y de control
• En sentido contrario ídem

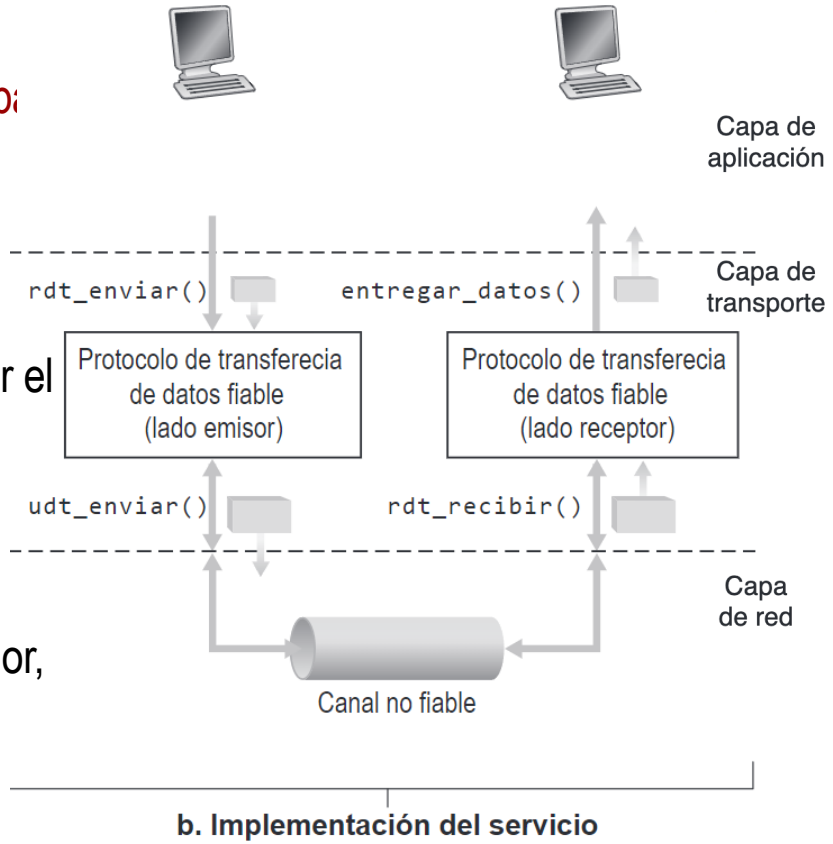
Transferencia de datos fiable: implementación del servicio.

EMISOR

- Llamada a "**rdt_send ( )**".
  - Invocación del "rdt emisor" por la capa superior (capa de Aplicación).
  - Pasa "datos" para ser enviados.
- Llamada a "**udt_send ( )**".
  - Invocación del canal (capa inferior, capa de Red) por el "rdt emisor".
  - Pasa "datos" a enviar al canal.

RECEPTOR

- Llamada a "**rdt_rcv ( )**".
  - Invocación del "rdt receptor" por el canal (capa inferior, capa de Red).
  - Toma "datos" recibidos del canal.
- Llamada a "**deliver_data**".
  - Invocación de la capa superior (capa de Aplicación) por el "rdt receptor".
  - Entrega "datos" recibidos.



b. Implementación del servicio

Transmisión:
- De paquetes de datos y de control
- En sentido contrario ídem

udt: **u**nreliable **d**ata **t**ransfer
rdt: **r**eliable **d**ata **t**ransfer

Transferencia de datos fiable: rdt 1.0.

- rdt 1.0: canal 100 % fiable.
  - No errores de bits.
  - No pérdida de paquetes.
- Receptor:
  - Puede recibir datos a cualquier velocidad, y
  - No requiere notificar al Emisor ➔ No asentimientos.
- Pero la realidad es bien distinta…
  - Errores de bits, y/o
  - Pérdida de paquetes.

Transferencia de datos fiable: errores de bits.

- Errores de bits.
  - Normalmente se producen en los elementos físicos, cuando un paquete:
    - Se transmite.
      - Trasvase del Emisor al canal de transmisión.
    - Se propaga
      - Trasvase por el canal de transmisión.
    - Accede a un *buffer*.
      - Trasvase del canal de transmisión al Receptor
- Por tanto, se requieren capacidades para gestionar errores de bits.

Transferencia de datos fiable: errores de bits.

- Capacidades para gestión de errores de bits en paquetes de datos
  - Protocolos *ARQ (Automatic Repeat Request)*
    - Se basan en la retransmisión automática de unidades de datos erróneas
    - Se requieren tres capacidades de protocolo adicionales para gestionar la presencia de errores de bits:

1. Detección de errores.
   - Mediante información redundante (*checksum*).
2. Asentimientos explícitos (realimentación del Receptor al Emisor).
   - ACK y NAK.
     - ACK para reconocer paquetes de datos NO corruptos.
     - NAK para reconocer paquetes de datos corruptos.
   - Por tanto, operación mediante *Stop – and – Wait Protocol* (Protocolo de parada y espera)
3. Retransmisión.
   - Como procedimiento para corrección de errores ➔ Corrección "hacia atrás".
     - ARQ: **A**utomatic **R**epeat re**Q**uest.

ACK: **ACK**nowledgement
NAK (NACK): **N**egative **ACK**

Transferencia de datos fiable: rdt 2.0.

- rdt 2.0: canal NO fiable.
  - Errores de bits y NO pérdida de paquetes.
- En principio, protocolo fiable mediante:
  - *Checksum*.
  - *Stop and Wait*.
  - *Automatic Repeat reQuest* (ARQ).
- ¡¡¡PERO…!!!
  - ¿Qué sucede si los paquetes ACK/NAK se corrompen? ¿Cómo se gestiona?
  - Si se retransmite ➔ Duplicados.
  - ¿Cómo el receptor detecta duplicados?

Transferencia de datos fiable: rdt 2.1.

- rdt 2.1: versión mejorada de rdt 2.0.
  - Errores de bits y NO pérdida de paquetes.
- Posibilita gestión de paquetes ACK/NAK corruptos.
  - Mediante "números de secuencia" en paquetes de datos.
    - Requiere campo de control para el Nº de secuencia.
      - Basta con un bit (protocolo *Stop and Wait*).
      - NO requerido en paquetes ACK/NAK.
    - Receptor capacitado para detectar paquetes de datos duplicados (retransmitidos).
- Receptor.
  - ¿Qué hace con los paquetes de datos duplicados?
  - No sabe si el último ACK/NAK llegó bien al emisor.

Transferencia de datos fiable: rdt 2.2.

- rdt 2.2: versión mejorada de rdt 2.1.
  - Errores de bits y NO pérdida de paquetes.
  - Soluciona problema de paquetes de datos duplicados.
- No emplea NAK, SÓLO paquetes ACK.
  - ACK con "Nº de secuencia".
    - Nº de secuencia del paquete de datos que esté siendo confirmado.
  - ACK repetidos produce mismo efecto que NAK ➔ Retransmisión.
- Nuevas funciones para el Receptor y el Emisor.
  - Receptor.
    - Añadir Nº de secuencia a los ACK - Paquetes duplicados se descartan.
  - Emisor.
    - Comprobar "Nº de secuencia" de los ACK que recibe.

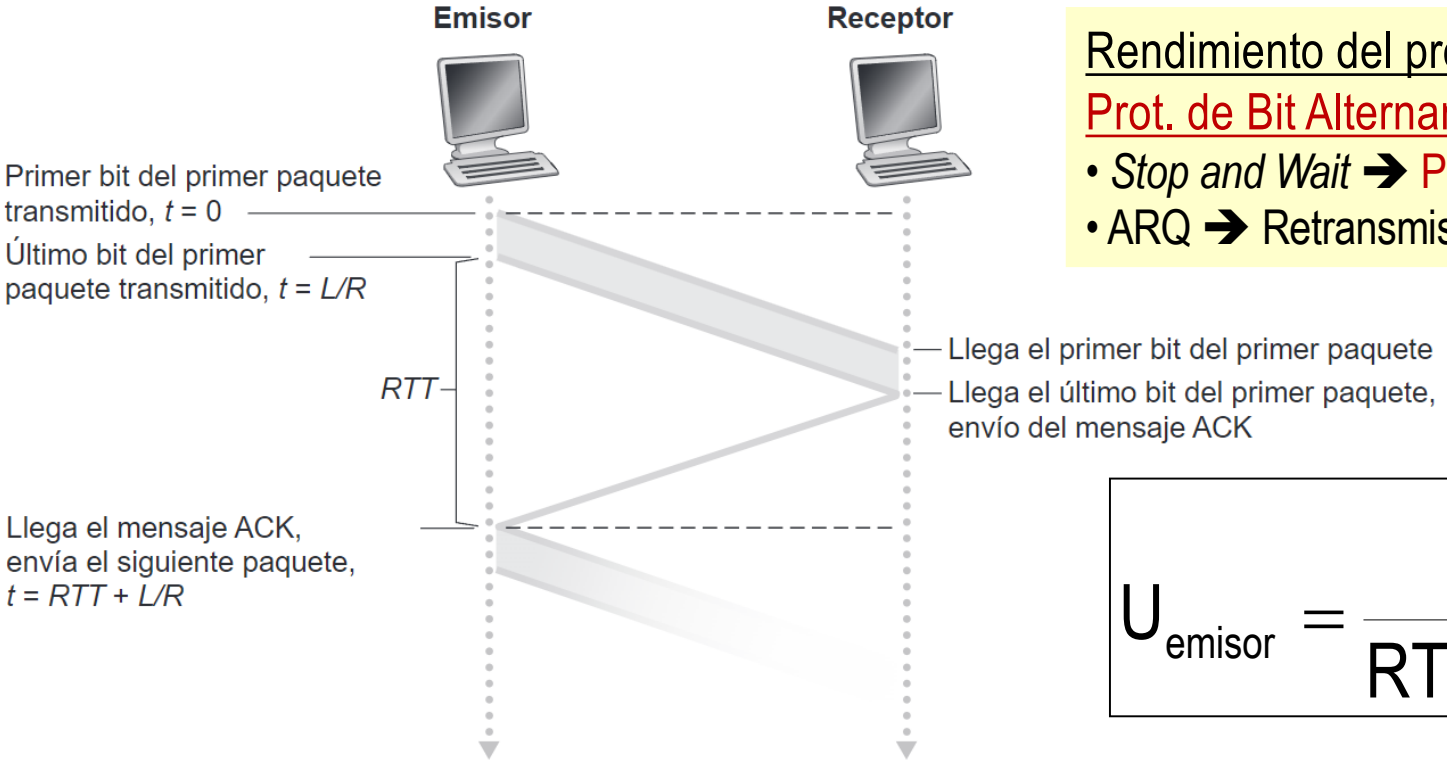
Transferencia de datos fiable: rdt 3.0.

- rdt 3.0: versión mejorada de rdt 2.2.
  - Errores de bits y pérdida de paquetes.
  - Contexto real, canales que pueden corromper y "perder" paquetes.
- Pérdida de paquetes (de datos y/o de control).
  - Con rdt 2.2 Emisor NO se entera.
- Dos problemas más para el Emisor:
  - Detectar pérdida de paquetes, y
  - Gestionar la pérdida de paquetes.

Transferencia de datos fiable: rdt 3.0.

- Solución a los dos problemas anteriores:
  - Esperar un tiempo "prudencial" por los ACK, si NO llegan ➔ Retransmitir.
  - ¿Cuánto tiempo esperar? En general muy difícil de estimar. En la práctica:
    - Emisor inicia un temporizador cuando (re)transmite un paquete.
    - Emisor requiere nuevas funciones ➔ Gestión de temporizadores.
- Denominado en ocasiones PROTOCOLO DE BIT ALTERNANTE.
  - Dada la alternancia entre "1" y "0" de los números de secuencia.
  - Contempla los elementos claves de un protocolo de transferencia de datos fiable:
    - Sumas de comprobación ➔ Detección de errores, rdt2.0.
    - ARQ ➔ Corrección de errores, rdt2.0.
    - ACK y NAK (ACK repetidos) ➔ Asentimientos, rdt2.0 (rdt2.2).
    - Nº de secuencia ➔ Detección de paquetes de datos duplicados, rdt2.1.
    - Temporizadores ➔ Detección de pérdida de paquetes, rdt3.0.

Transferencia de datos fiable: rdt 3.0.



Rendimiento del protocolo rdt 3.0

Prot. de Bit Alternante

- *Stop and Wait* ➔ Problema de rendimiento
- ARQ ➔ Retransmisión para corregir errores

$$U_{emisor} = \frac{L/R}{RTT + L/R}$$

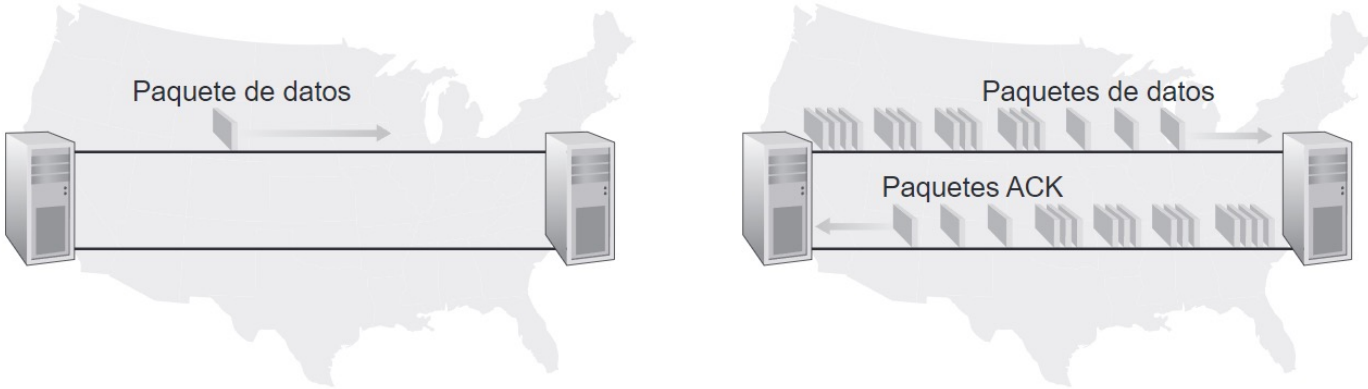
- Tasa de utilización del canal (U_{emisor}):
  - P.e: si RTT=30ms, R=1Gbps y L=1000Bytes ➔ $U_{emisor} \cong$ **0,000267**

    Es decir, una tasa efectiva aproximada de 267 Kbps en un canal de 1Gbps.

¡El protocolo limita el uso de recursos físicos!

Protocolos fiables con Procesamiento en Cadena.

- PROCESAMIENTO EN CADENA (*pipelining*) ← Para mejorar rendimiento del protocolo S&W
  - Múltiples paquetes pendientes de reconocimiento.

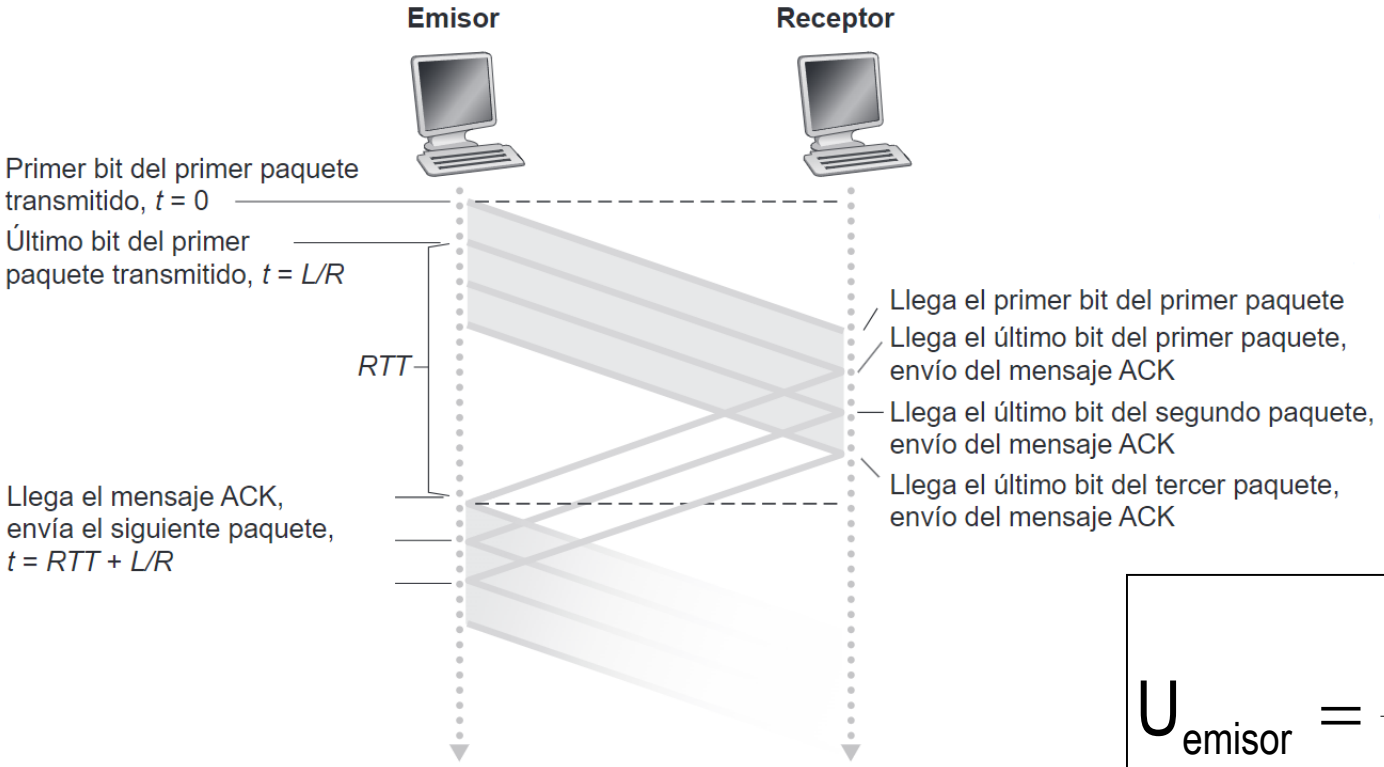


a. Funcionamiento de un protocolo de parada y espera

b. Funcionamiento de un protocolo con procesamiento en cadena

- Consecuencias:
  - Incremento del rango de números de secuencia.
  - Almacenamiento (*buffering*) en Emisor, y a veces en el Receptor.*
- Dos estrategias básicas:
  - Retroceder N (*Go-Back-N*, GBN). También denominado "repetición NO selectiva".
  - Repetición Selectiva (*Selective Repeat*, SR).

Procesamiento en Cadena: rendimiento.



$$U_{emisor} = \frac{k\,L/R}{RTT + L/R}$$

- Tasa de utilización del canal (U_{emisor}).
  - En principio, la utilización se multiplica por k.
    - k: número de paquetes que se envían en cadena.

Procesamiento en Cadena: desafíos.

- Desafíos.
  - ¿Cuántos paquetes de datos pendientes de ACK se permiten?
  - ¿Cómo se indica la recepción errónea de un paquete?
  - ¿Cómo reacciona el Emisor cuando se corrompe un paquete?

Protocolos con procesamiento en "cadena": variantes.

Retroceder-N (GBN)

- Emisor
  - Admite hasta N paquetes sin confirmar.
  - Temporizador para el paquete sin confirmar más antiguo.
    - Si expira el Temporizador:
      - Se retransmiten todos los paquetes sin confirmar.
- Receptor.
  - Envía ACKs acumulativos.
  - No asiente paquetes fuera de orden.

Repetición selectiva (SR)

- Emisor
  - Admite hasta N paquetes sin confirmar
  - Temporizador para cada paquete sin confirmar.
    - Si expira un temporizador:
      - Se retransmite sólo el paquete afectado.
- Receptor
  - Envía ACKs no acumulativos.
    - ACKs por cada paquete.
  - Sí asiente paquetes fuera de orden.